```python
#finding the middle node

class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)
node5 = Node(50)

node1.next = node2
node2.next = node3
node3.next = node4
node4.next = node5

slow = node1
fast = node1
while fast is not None and fast.next is not None:
  slow = slow.next
  fast = fast.next.next
print("Middle node: ", slow.data)
```

```
Middle node:  30
```

Day 5

LIFO v/s FIFO:

FIFO	LIFO
First-In-First-Out	Last-In-First-Out
New element is inserted below the existing element. Oldest element is at the top and taken out first	New element is inserted above the existing element. Newest element is at the top and taken out first

First element to be entered gets out first	The first element to be entered is the last to get out.
Time complexity of inserting an element is **O(1)**	**Time complexity** of inserting an element is **O(1)**
Implemented in **queue** data structure	Implemented in **stack** data structure

Stack operations:

1. **push()** —> to insert an element into the stack
2. **pop()** —> to remove an element from the stack
3. **top()** —> returns the top element of the stack
4. **isEmpty()** —> returns True if stack s empty, else False
5. **size()** —> returns the size of the stack

Applications of stack -

- Function calls
- Recursion
- Syntax parsing
- Memory management

Queue operations:

1. enqueue() —> insertion of elements to the queue
2. dequeue() —> removal of elements from the queue
3. getFront() —> acquires the data element at the front node without deleting it
4. getRear() —> returns the element at the rear end without removing it
5. isEmpty() —> returns True if stack s empty, else False
6. size() —> returns the size of the queue

Applications of queue -

- CPU scheduling
- Job scheduling
- Networking
- Buffer management

Deque:

It stands for Double-Ended Queue. It is a data structure that allows to add and remove elements from both ends efficiently.

- Allows fast insertion and deletion from both front and rear ends

- ***Works as both stack (LIFO) and queue (FIFO)***

Operations on deque:

1. **append()** —> adds an element to the right end of the dequeue
2. **appendleft()** —> adds an element to the left of the dequeue
3. **extend()** —> adds multiple elements to the right end of the deque
4. **extendleft():** adds multiple elements to the left end of the deque
5. **remove():** removes the first occurrence of a specified value
6. **pop()** —> removes and returns the element from the right end
7. **popleft()** —> removes and returns the element from the left end
8. **clear()** —> removes all elements from the deque
9. **len()** —> returns the total number of elements in the deque
10. **count()** —> returns how many times a specific element appears in the deque
11. **reverse()** —> reverses the order of elements in the deque

Creating stack using a python list:

```
#creating stack using list

class Stack:
  def __init__(self):
    self.values = []
  def push(self, x):
    self.values = [x] + self.values
  def pop(self):
    return self.values.pop(0)

s = Stack()
s.push(10)
s.push(20)
s.push(30)
s.push(40)
print(s.values)
```

```
[40, 30, 20, 10]
```

Creating queue using deque:

```
#creating queue using dequeue

from collections import deque
d = deque([1, 2, 3, 4, 5])
print(d)
```

```
deque([1, 2, 3, 4, 5])
```

Reversing a stack:

```
#reversing a stack

class Stack:
  def __init__(self):
    self.values = []
  def push(self, x):
    self.values = [x] + self.values
  def pop(self):
    return self.values.pop(0)
  def reverse(self):
    self.values = self.values[::-1]

s = Stack()
s.push(10)
s.push(20)
s.push(30)
s.push(40)
print("before reversing: ", s.values)
s.reverse()
print("after reversing: ", s.values)
```

```
before reversing:  [40, 30, 20, 10]
after reversing:  [10, 20, 30, 40]
```